AI600 – Deep Learning

Assignment 3: Analytical Solutions

Muhammad Sami Ul Basit
Roll No: 25280025
github.com/msbasit/AI-600-Assignment-3
LUMS – School of Science and Engineering

Spring 2026

Contents

Notation and Setup **3**

1 Question 1: Forward and Backward Pass **4**
1a) Forward Pass . . . 4
1b) Backward Pass . . . 5
1c) Interpretation . . . 6

2 Question 3: Pooling and Normalization **8**
3a) Max Pooling . . . 8
3b) Layer Normalization . . . 8
3c) Interpretation: Translation Invariance . . . 9

Programming Component **10**

3 Task 1: Custom CNNs and Shortcut Learning **10**
3.1 Task 1A – Standard MNIST . . . 10
Model Architecture . . . 10
Training Setup . . . 10
Training Log . . . 11
Learning Curves . . . 11
Analytical Question 1.1 . . . 11
First-Layer Filter Visualisation . . . 12
Analytical Question 1.2 . . . 12
3.2 Task 1B – Colored MNIST (C-MNIST) . . . 12
Dataset and Adaptation . . . 12
Sample Images . . . 13
Training Log . . . 13
Evaluation Results . . . 13
Analytical Question 1.3 . . . 13
Analytical Question 1.4 . . . 14

4 Task 2: Transfer Learning and Interpretability **16**
4.1 Task 2A – Fine-tuning ResNet-18 on STL-10 . . . 16
Setup . . . 16
Training Log . . . 16
Analytical Question 2.1 . . . 16
4.2 Task 2B – GradCAM Visualisation . . . 17
Implementation . . . 17
GradCAM Heatmaps . . . 17
Analytical Question 2.2 . . . 17
Analytical Question 2.3 . . . 18

Summary of Results **20**

Notation and Setup

Before working through the questions, let us fix the values given in the problem statement.

$$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}, \quad b = 0$$

$$V = \begin{pmatrix} 1 & -1 & 2 & 0.5 \end{pmatrix}, \quad c = 1, \quad T = 3$$

The architecture performs the following operations in order:

1. **Convolution**: $Z = X * W + b$ (valid convolution, stride 1, no padding, gives a 2×2 output)
2. **ReLU**: $A = \mathrm{ReLU}(Z)$
3. **Flatten**: $A_{\text{flat}} = [a_{11},\, a_{12},\, a_{21},\, a_{22}]^T \in \mathbb{R}^4$
4. **Fully Connected**: $Y = V \cdot A_{\text{flat}} + c$
5. **Loss**: $L = \frac{1}{2}(Y - T)^2$

1 Question 1: Forward and Backward Pass

1a) Forward Pass

Step 1 – Pre-activation feature map Z. A valid convolution with a 2×2 filter on a 3×3 input (stride 1) slides the filter over four positions, yielding a 2×2 output. Let Z_{ij} denote the element at row i, column j:

$$Z_{ij} = \sum_{m=0}^{1} \sum_{n=0}^{1} X_{i+m,\, j+n} \cdot W_{m+1,\, n+1}$$

Computing each element explicitly (with $b = 0$ throughout):

$$Z_{11} = X_{11}W_{11}+X_{12}W_{12}+X_{21}W_{21}+X_{22}W_{22} = (1)(1)+(2)(0)+(0)(-1)+(1)(1) = 1+0+0+1 = 2$$

$$Z_{12} = X_{12}W_{11}+X_{13}W_{12}+X_{22}W_{21}+X_{23}W_{22} = (2)(1)+(0)(0)+(1)(-1)+(1)(1) = 2+0-1+1 = 2$$

$$Z_{21} = X_{21}W_{11}+X_{22}W_{12}+X_{31}W_{21}+X_{32}W_{22} = (0)(1)+(1)(0)+(1)(-1)+(0)(1) = 0+0-1+0 = -1$$

$$Z_{22} = X_{22}W_{11}+X_{23}W_{12}+X_{32}W_{21}+X_{33}W_{22} = (1)(1)+(1)(0)+(0)(-1)+(2)(1) = 1+0+0+2 = 3$$

$$\boxed{Z = \begin{pmatrix} 2 & 2 \\ -1 & 3 \end{pmatrix}}$$

Step 2 – Post-activation feature map $A = \mathbf{ReLU}(Z)$. ReLU zeroes out negative entries and keeps positive ones unchanged:

$$\boxed{A = \begin{pmatrix} 2 & 2 \\ 0 & 3 \end{pmatrix}}$$

Element $Z_{21} = -1 < 0$ is clamped to zero; the other three pass through.

Step 3 – Final prediction Y. Flatten A row-major:

$$A_{\text{flat}} = \begin{pmatrix} 2 \\ 2 \\ 0 \\ 3 \end{pmatrix}$$

$$Y = V \cdot A_{\text{flat}} + c = (1)(2) + (-1)(2) + (2)(0) + (0.5)(3) + 1 = 2 - 2 + 0 + 1.5 + 1$$

$$\boxed{Y = 2.5}$$

Step 4 – Loss L.

$$L = \frac{1}{2}(Y - T)^2 = \frac{1}{2}(2.5 - 3)^2 = \frac{1}{2}(-0.5)^2 = \frac{1}{2}(0.25)$$

$$\boxed{L = 0.125}$$

1b) Backward Pass

We apply the chain rule, working from the loss back toward the weights.

Gradient of loss w.r.t. Y.

$$\frac{\partial L}{\partial Y} = Y - T = 2.5 - 3 = -0.5$$

Gradient of loss w.r.t. V $\left(\frac{\partial L}{\partial V}\right)$. Since $Y = V \cdot A_{\text{flat}} + c$, we have $\frac{\partial Y}{\partial V} = A_{\text{flat}}^T$. Therefore:

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial Y} \cdot A_{\text{flat}}^T = (-0.5)\begin{pmatrix} 2 & 2 & 0 & 3 \end{pmatrix}$$

$$\boxed{\frac{\partial L}{\partial V} = \begin{pmatrix} -1 & -1 & 0 & -1.5 \end{pmatrix}}$$

Gradient of loss w.r.t. A_{flat}.

$$\frac{\partial L}{\partial A_{\text{flat}}} = \frac{\partial L}{\partial Y} \cdot V^T = (-0.5)\begin{pmatrix} 1 \\ -1 \\ 2 \\ 0.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \\ -1 \\ -0.25 \end{pmatrix}$$

Reshaping back to a 2×2 matrix:

$$\frac{\partial L}{\partial A} = \begin{pmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{pmatrix}$$

Gradient of loss w.r.t. Z (through ReLU). ReLU passes the gradient only where its input was positive ($Z_{ij} > 0$). Define the indicator $\mathbf{1}[Z_{ij} > 0]$:

$$\frac{\partial L}{\partial Z_{ij}} = \frac{\partial L}{\partial A_{ij}} \cdot \mathbf{1}[Z_{ij} > 0]$$

The activation map Z had: $Z_{11} = 2 > 0$, $Z_{12} = 2 > 0$, $Z_{21} = -1 < 0$, $Z_{22} = 3 > 0$.

$$\frac{\partial L}{\partial Z} = \begin{pmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{pmatrix}$$

The entry for Z_{21} is zero because the ReLU gate was closed there.

Gradient of loss w.r.t. W $\left(\frac{\partial L}{\partial W}\right)$. For a convolution $Z = X * W$, the gradient w.r.t. the filter is the cross-correlation of the input X with the upstream gradient $\delta_Z = \frac{\partial L}{\partial Z}$:

$$\frac{\partial L}{\partial W_{mn}} = \sum_{i,j} \delta_{Z_{ij}} \cdot X_{i+m-1,\, j+n-1}$$

Writing it out for each weight position:

$$\frac{\partial L}{\partial W_{11}} = \delta_{Z_{11}} X_{11} + \delta_{Z_{12}} X_{12} + \delta_{Z_{21}} X_{21} + \delta_{Z_{22}} X_{22}$$
$$= (-0.5)(1) + (0.5)(2) + (0)(0) + (-0.25)(1) = -0.5 + 1 + 0 - 0.25 = \mathbf{0.25}$$

$$\frac{\partial L}{\partial W_{12}} = \delta_{Z_{11}} X_{12} + \delta_{Z_{12}} X_{13} + \delta_{Z_{21}} X_{22} + \delta_{Z_{22}} X_{23}$$
$$= (-0.5)(2) + (0.5)(0) + (0)(1) + (-0.25)(1) = -1 + 0 + 0 - 0.25 = \mathbf{-1.25}$$

$$\frac{\partial L}{\partial W_{21}} = \delta_{Z_{11}} X_{21} + \delta_{Z_{12}} X_{22} + \delta_{Z_{21}} X_{31} + \delta_{Z_{22}} X_{32}$$
$$= (-0.5)(0) + (0.5)(1) + (0)(1) + (-0.25)(0) = 0 + 0.5 + 0 + 0 = \mathbf{0.5}$$

$$\frac{\partial L}{\partial W_{22}} = \delta_{Z_{11}} X_{22} + \delta_{Z_{12}} X_{23} + \delta_{Z_{21}} X_{32} + \delta_{Z_{22}} X_{33}$$
$$= (-0.5)(1) + (0.5)(1) + (0)(0) + (-0.25)(2) = -0.5 + 0.5 + 0 - 0.5 = \mathbf{-0.5}$$

$$\boxed{\frac{\partial L}{\partial W} = \begin{pmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{pmatrix}}$$

1c) Interpretation – Which Weight Changes the Most?

Looking at the absolute values of the gradient entries:

$$\left|\frac{\partial L}{\partial W}\right| = \begin{pmatrix} 0.25 & \mathbf{1.25} \\ 0.5 & 0.5 \end{pmatrix}$$

The weight W_{12} (top-right entry, originally 0) has the largest absolute gradient of 1.25. A gradient descent update $W \leftarrow W - \eta \nabla_W L$ would decrease W_{12} by $-\eta(-1.25) = +1.25\eta$, so W_{12} *increases* after one step.

Structural explanation. The gradient for W_{12} accumulates contributions from the input pixels it multiplied in each valid position of the convolution:

$$\frac{\partial L}{\partial W_{12}} = \sum_{(i,j)} \delta_{Z_{ij}} \cdot X_{i,\, j+1} = \delta_{Z_{11}} X_{12} + \delta_{Z_{12}} X_{13} + \delta_{Z_{21}} X_{22} + \delta_{Z_{22}} X_{23}$$

The dominant terms are $\delta_{Z_{11}} X_{12} = (-0.5)(2) = -1$ and $\delta_{Z_{22}} X_{23} = (-0.25)(1) = -0.25$. Note that $X_{12} = 2$ is the largest pixel value in the positions that W_{12} ever sees during the convolution. Larger activation inputs create a proportionally larger gradient for the corresponding weight, because in the chain rule the input acts as the "local derivative" of the pre-activation output w.r.t. the weight. Additionally, the only ReLU-gated-off contribution ($\delta_{Z_{21}} = 0$) is the one that would have partially cancelled this large negative sum, so the gate being closed amplifies the net magnitude.

2 Question 3: Pooling and Normalization

3a) Max Pooling

We replace flattening and the FC layer with a 2×2 Global Max Pool applied to A:

$$A = \begin{pmatrix} 2 & 2 \\ 0 & 3 \end{pmatrix}$$

$$\boxed{Y_{\text{pool}} = \max(A) = 3}$$

The maximum is attained at position $(2, 2)$, i.e. $A_{22} = 3$.

Gradient flow comparison. In the original architecture, *all four* non-zero activations (positions 11, 12, and 22) contributed to Y through their respective V weights. The gradient flowed back from every active position to every input pixel that contributed to those positions.

With global max pooling, the gradient flows back *only* through the winning position:

$$\frac{\partial Y_{\text{pool}}}{\partial A_{ij}} = \begin{cases} 1 & \text{if } (i, j) = \text{argmax of } A \\ 0 & \text{otherwise} \end{cases}$$

Here argmax $= (2, 2)$, so the gradient is 1 at A_{22} and 0 everywhere else.

Tracing this back through ReLU and into X: only the four input pixels that contributed to Z_{22} – namely $X_{22}, X_{23}, X_{32}, X_{33}$ – receive a non-zero gradient. The other five pixels $(X_{11}, X_{12}, X_{13}, X_{21}, X_{31})$ all have **zero gradient**.

In the original architecture $A_{21} = 0$ was already gated off by ReLU, so the input pixels exclusively feeding Z_{21} (i.e. $X_{21}, X_{22}, X_{31}, X_{32}$, exclusive to Z_{21}) received zero gradient there too; but pixels such as X_{12} that fed multiple Z positions still received non-zero gradients. Max pooling makes this sparsity much more aggressive.

Which pixels in X have zero gradient under max pooling? Z_{22} is computed from: $X_{22}, X_{23}, X_{32}, X_{33}$. These four receive non-zero gradient. All other pixels of X (i.e. $X_{11}, X_{12}, X_{13}, X_{21}, X_{31}$) have gradient zero.

3b) Layer Normalization

We return to the original architecture and insert Layer Norm after ReLU, applied to the 2×2 feature map A:

$$A = \begin{pmatrix} 2 & 2 \\ 0 & 3 \end{pmatrix}$$

Flatten to compute statistics over all four elements $\{2, 2, 0, 3\}$:

Mean μ:

$$\mu = \frac{2 + 2 + 0 + 3}{4} = \frac{7}{4} = 1.75$$

Variance σ^2:

$$\sigma^2 = \frac{1}{4}\left[(2-1.75)^2 + (2-1.75)^2 + (0-1.75)^2 + (3-1.75)^2\right]$$

$$= \frac{1}{4}\left[(0.25)^2 + (0.25)^2 + (-1.75)^2 + (1.25)^2\right] = \frac{1}{4}\left[0.0625 + 0.0625 + 3.0625 + 1.5625\right] = \frac{4.75}{4} = \mathbf{1.1875}$$

Normalized map A_{norm} (with $\gamma = 1$, $\beta = 0$, $\epsilon = 0$):

$$A_{\text{norm},\,ij} = \frac{A_{ij} - \mu}{\sigma} = \frac{A_{ij} - 1.75}{\sqrt{1.1875}}$$

$$\sigma = \sqrt{1.1875} \approx 1.0898$$

$$\boxed{A_{\text{norm}} = \begin{pmatrix} \frac{2-1.75}{1.0898} & \frac{2-1.75}{1.0898} \\ \frac{0-1.75}{1.0898} & \frac{3-1.75}{1.0898} \end{pmatrix} \approx \begin{pmatrix} 0.2294 & 0.2294 \\ -1.6059 & 1.1471 \end{pmatrix}}$$

By construction, A_{norm} has mean 0 and variance 1 over all four elements.

3c) Max Pooling and Translation Invariance

Translation invariance from max pooling arises from the following structural argument.

Suppose we shift the input image X slightly to the left. In a CNN with multiple convolutional layers, a small spatial shift in the input tends to produce a corresponding small spatial shift in the intermediate feature maps. Concretely, if activation $A_{22} = 3$ is the maximum, a one-pixel shift to the left might cause the value 3 to move to position A_{21} instead.

With global max pooling, the output is $\max(A)$ – it does not care *which position* holds the maximum, only *what that maximum value is*. Therefore:

$$Y_{\text{pool}} = \max_{i,j} A_{ij}$$

As long as the shift is small enough that the maximum value itself does not change (it just moves around spatially within the feature map), the output Y and consequently the loss L remain the same. Mathematically, if after a small shift $A' = \text{shift}(A)$ satisfies $\max(A') = \max(A)$, then $Y' = Y$ exactly.

This is in contrast to a flattening-based FC approach, where the output depends on both the values *and* the specific positions of the activations (because different positions are weighted by different V_i). A spatial shift would rearrange which value is multiplied by which weight, producing a different Y even for identical feature values.

Programming Component

3 Task 1: Custom CNNs and Shortcut Learning

3.1 Task 1A – Standard MNIST

Model Architecture

A compact CNN was implemented in PyTorch with the following structure, staying well within all specified constraints:

Layer	Configuration	Parameters
Conv2d (Block 1)	1→8 channels, 3×3, valid	80
ReLU + MaxPool2d	2×2 stride 2	–
Conv2d (Block 2)	8→16 channels, 3×3, valid	1,168
ReLU + MaxPool2d	2×2 stride 2	–
Conv2d (Block 3)	16→32 channels, 3×3, same padding	4,640
ReLU + AdaptiveAvgPool2d	global 1×1	–
Linear	$32 \to 64$	2,112
ReLU + Dropout(0.3)		–
Linear	$64 \to 10$	650
Total		**8,650**

The model uses GlobalAveragePooling after the third convolutional block, which collapses the spatial dimensions to a single 32-dimensional vector before the fully connected layers. This design choice avoids a large flattening dimension and keeps the parameter count far below the 50 000 limit.

Training Setup

- **Optimiser:** Adam, learning rate 10^{-3}, weight decay 10^{-4}
- **Loss:** Cross-Entropy
- **Scheduler:** StepLR – halve LR every 5 epochs
- **Epochs:** 15
- **Splits:** 54 000 train / 6 000 validation / 10 000 test

Training Log

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
01	1.2214	0.4862	56.94%	86.18%
02	0.4256	0.2734	87.13%	91.72%
03	0.2995	0.2133	91.06%	93.38%
04	0.2475	0.1734	92.69%	94.83%
05	0.2160	0.1721	93.71%	94.60%
06	0.1857	0.1494	94.61%	95.50%
07	0.1792	0.1444	94.67%	95.70%
08	0.1642	0.1316	95.18%	96.00%
09	0.1580	0.1308	95.38%	96.12%
10	0.1494	0.1161	95.61%	96.53%
11	0.1378	0.1132	96.00%	96.53%
12	0.1336	0.1136	96.02%	96.50%
13	0.1325	0.1035	96.20%	96.83%
14	0.1252	0.1031	96.28%	96.92%
15	0.1229	0.1031	96.44%	96.95%
Final Test Accuracy				**97.31%**

Learning Curves

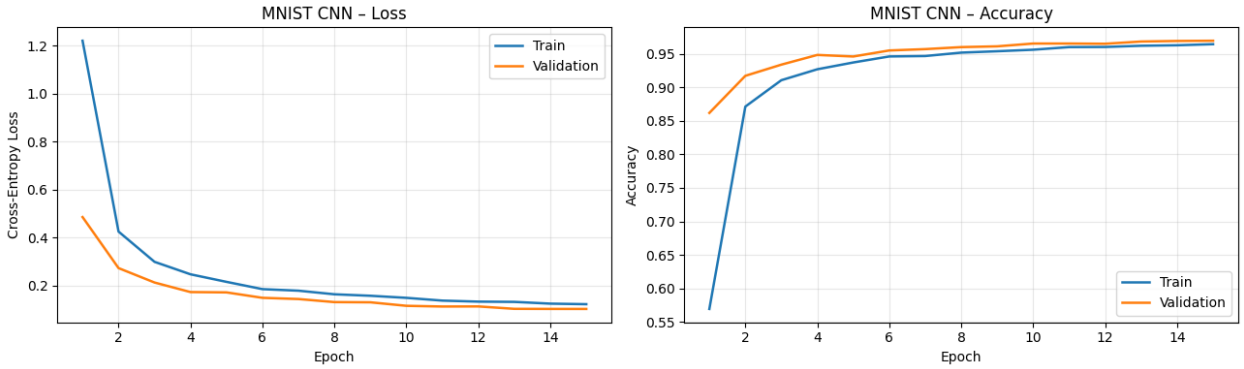

Figure 1: MNIST training and validation loss (left) and accuracy (right) over 15 epochs.

Analytical Question 1.1 – Overfitting, Underfitting, or Good Generalisation?

Looking at the loss and accuracy curves, the training and validation lines track each other closely throughout all 15 epochs. The final training accuracy (96.44%) and validation accuracy (96.95%) are within roughly 0.5 percentage points of each other, and both loss curves decrease together without the validation curve turning back upward. This is the signature of a model that **generalises well**: there is no growing gap between the two curves, which would indicate overfitting, and the model is not stuck at a high loss, which would indicate underfitting. The small parameter count (< 50k), the Dropout layer, and the weight decay in Adam all work together to keep the model from memorising the training set.

First-Layer Filter Visualisation

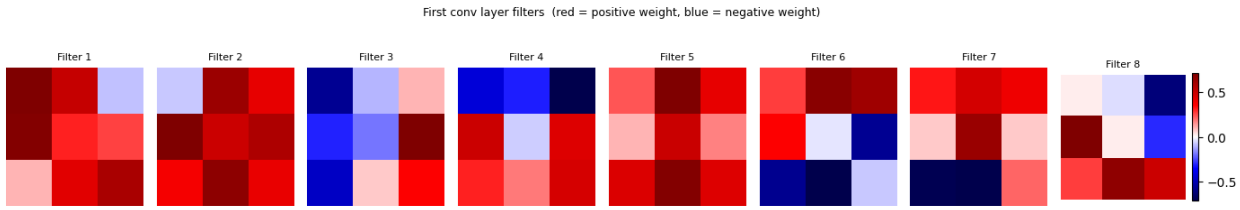

Figure 2: The 8 learned filters from the first convolutional layer. Red = positive weight, blue = negative weight (seismic colourmap).

Analytical Question 1.2 – What are the Filters Detecting?

Even with only 3×3 kernels, rudimentary visual patterns are visible in the filters after training. Filters that have a positive region on one side and a negative region on the other act as **edge detectors**: when such a filter slides over an image, it responds strongly wherever pixel intensity transitions sharply from dark to light (or vice versa). Some filters show a centre-positive / surround-negative pattern, similar to a Laplacian, which responds to blobs or corners. Others appear more diffuse, possibly acting as low-frequency smoothers that carry average brightness information. These behaviours are consistent with what is known about early convolutional layers in well-trained CNNs: they learn to detect basic local structure (edges, textures, orientations) that higher layers then combine into class-discriminative representations.

3.2 Task 1B – Colored MNIST (C-MNIST)

Dataset and Adaptation

The C-MNIST dataset provides 60 000 training images of shape $(3, 28, 28)$ in which each digit class is systematically paired with a specific background colour in the biased split, while the unbiased test set breaks this pairing. The MNIST CNN was adapted by changing only the first convolutional layer from 1 input channel to 3 (RGB), keeping all other hyperparameters identical. Final model parameter count: **8 794**.

Figure 3: Top row: one sample per class from the biased training set – each digit has a fixed colour. Bottom row: corresponding samples from the unbiased test set – digit–colour pairing is randomised.

Sample Images

Training Log

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
01	0.7884	0.4109	82.03%	93.70%
02	0.3824	0.3348	94.16%	94.48%
03	0.3354	0.3037	94.81%	94.57%
04	0.3016	0.2826	94.95%	94.62%
05	0.2735	0.2526	95.10%	94.83%
06	0.2473	0.2326	95.30%	94.97%
07	0.2296	0.2255	95.41%	95.07%
08	0.2132	0.2026	95.51%	95.18%
09	0.1979	0.1847	95.66%	95.40%
10	0.1808	0.1649	95.81%	95.62%
11	0.1641	0.1568	96.04%	95.72%
12	0.1555	0.1509	96.14%	95.75%
13	0.1501	0.1419	96.19%	95.98%
14	0.1429	0.1363	96.32%	96.08%
15	0.1361	0.1305	96.44%	96.32%

Evaluation Results

Test Set	**Accuracy**
Biased test set	96.75%
Unbiased test set	36.64%
Performance drop	**60.11 pp**

Analytical Question 1.3 – Why the Large Drop on the Unbiased Test Set?

The model is trained on a dataset where every digit class is perfectly correlated with a specific colour. From the perspective of the loss minimiser (Adam with cross-entropy), colour is an

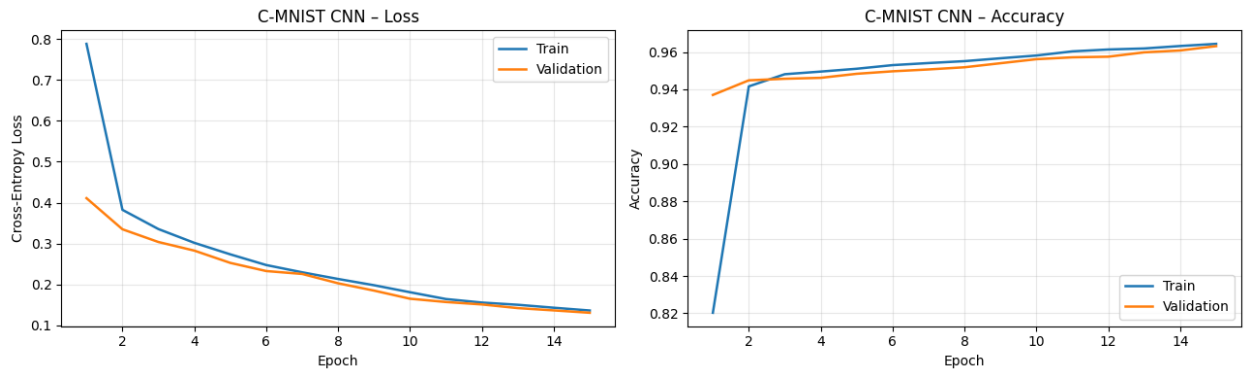

Figure 4: C-MNIST training and validation loss (left) and accuracy (right) over 15 epochs.

extremely easy feature to learn: it can be extracted by the very first convolutional layer in a single pass, and it provides nearly perfect classification on the training distribution. Shape, on the other hand, requires building up hierarchical representations across multiple layers, and its gradients are weaker because shape is sufficient but not the easiest path to low training loss.

Mathematically, the decision boundary learned by the network encodes:

$$P(y = c \mid x) \approx P(y = c \mid \text{colour}(x))$$

because the colour feature is essentially a lookup table: class c has colour k_c with probability ≈ 1 in the training set. Any filter that responds to that colour receives a large gradient signal that drives its weights far from zero. Shape-based filters also receive gradient, but the colour signal dominates because both features are equally predictive during training and colour is much simpler to represent.

On the unbiased test set the colour–class pairing is broken. The model's decision boundary, which was primarily colour-based, is no longer correlated with the true labels, and accuracy collapses from 96.75% to 36.64%. This is a canonical instance of **shortcut learning**: the network exploits a spurious correlation present in the training distribution rather than learning the causal feature (digit shape) that generalises to new distributions.

Analytical Question 1.4 – Training Strategies to Force Shape Learning

Several approaches can discourage shortcut learning on colour:

1. **Greyscale augmentation** – Randomly convert images to grayscale (and back to 3-channel) with some probability p during training. This forces the model to maintain shape-based representations that still work when colour is absent.

2. **Colour jitter augmentation** – Apply random hue, saturation, and brightness perturbations so that the fixed digit–colour mapping is disrupted during training. With strong enough jitter, colour becomes an unreliable cue and the network must rely on shape.

3. **Learning Invariant Predictors (IRM)** – A principled regularisation method that penalises any classifier whose gradient is non-zero on a dummy invariant classifier. It explicitly discourages features that work only within one environment (colour-paired) rather than across all environments.

4. **Resampling / reweighting** – If a held-out unbiased validation set is available, up-weight examples where the spurious correlation is absent so that the loss landscape favours shape-based features.

5. **Adversarial colour removal head** – Add a secondary classifier trained to predict colour and negate its gradient before it flows into the shared backbone (gradient reversal layer). This explicitly removes colour information from the learned representation.

4 Task 2: Transfer Learning and Interpretability

4.1 Task 2A – Fine-tuning ResNet-18 on STL-10

Setup

- **Base model:** ResNet-18 pre-trained on ImageNet (`ResNet18_Weights.IMAGENET1K_V1`)
- **Backbone:** All convolutional layers frozen (11,176,512 frozen parameters)
- **Head:** Original 1000-class FC replaced with a new `Linear(512, 10)` layer (5,130 trainable parameters)
- **Input:** STL-10 images resized from 96×96 to 224×224 and normalised with ImageNet statistics
- **Optimiser:** Adam on head only, LR 10^{-3}, StepLR decay
- **Epochs:** 10

Training Log

Epoch	Train Loss	Train Acc
01	0.7496	81.64%
02	0.2741	93.02%
03	0.2181	93.68%
04	0.1806	94.76%
05	0.1711	94.88%
06	0.1609	95.12%
07	0.1490	95.66%
08	0.1504	95.52%
09	0.1441	95.86%
10	0.1361	96.12%
STL-10 Test Accuracy		**94.92%**

Analytical Question 2.1 – Why Freeze the Backbone?

There are two reasons, one computational and one functional.

Computationally, freezing the backbone means gradients do not need to be computed or stored for the vast majority of parameters (over 11 million in ResNet-18, versus 5 130 in the new FC layer). This dramatically reduces memory usage and speeds up each training step, which matters when the dataset is small.

Functionally, the early layers of a CNN trained on ImageNet have learned to detect general visual primitives: oriented edges (Gabor-like filters), colour blobs, and simple textures in the shallowest layers; progressively more complex patterns (corners, curves, object parts) in the middle layers. These representations are largely dataset-agnostic and transfer well

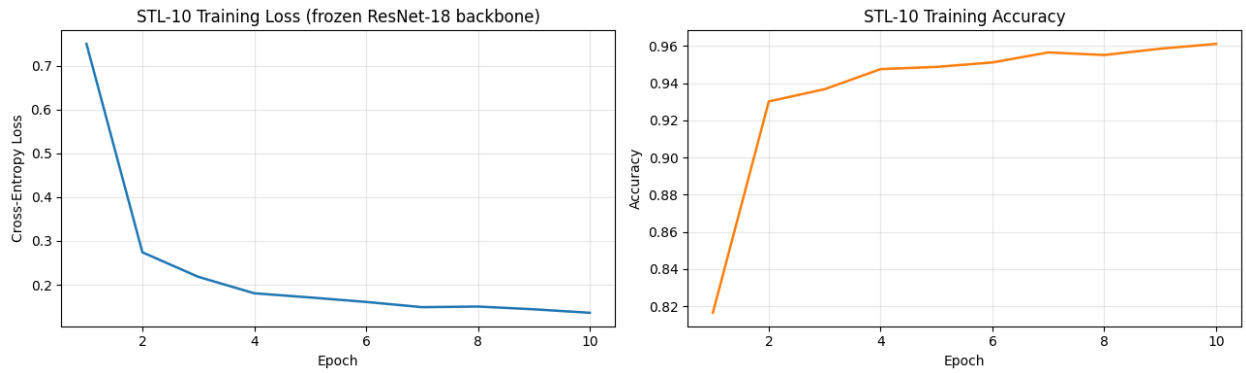

Figure 5: STL-10 training loss (left) and accuracy (right) over 10 epochs. Only the classification head is trained; the backbone is frozen.

to any natural-image classification task. The later layers are more task-specific and encode ImageNet class structure, which is less useful for STL-10.

Fine-tuning all layers on a small dataset like STL-10 (5 000 training images) would risk catastrophic forgetting of the general low-level features and lead to overfitting, because there are far more parameters than needed to explain the small training distribution. By keeping the backbone frozen, we effectively use it as a fixed feature extractor and only teach the last layer the STL-10 class boundaries.

4.2 Task 2B – GradCAM Visualisation

Implementation

GradCAM was implemented from scratch using PyTorch hooks. A forward hook on `resnet.layer4` captures the activation tensor $A^k \in \mathbb{R}^{C\times H'\times W'}$. Because the backbone is frozen (`requires_grad=False`), `.retain_grad()` is called on the activation tensor inside the forward hook to force the gradient to be retained after `backward()`. The importance weights and spatial map are then:

$$\alpha_k = \frac{1}{H'W'}\sum_{i,j}\frac{\partial y_c}{\partial A^k_{ij}}, \qquad \text{CAM} = \text{ReLU}\left(\sum_k \alpha_k A^k\right)$$

The resulting 7×7 map is bilinearly upsampled to 224×224 and overlaid on the input image.

GradCAM Heatmaps

Analytical Question 2.2 – Correct Prediction Heatmaps

For the two correctly classified images, the GradCAM heatmap concentrates its high-activation region (red and yellow in the jet colourmap) on the **primary object**. For example, if the true class is "airplane", the hottest pixels fall over the fuselage and wings rather than the sky background. For "dog", activation concentrates around the animal's head or body.

Figure 6: GradCAM visualisation for 4 STL-10 test images. **Top row:** original image with true and predicted class. **Middle row:** raw GradCAM heatmap (red/yellow = high importance, blue = low). **Bottom row:** heatmap overlaid on the original image. The first two columns are correct predictions; the last two are incorrect.

This is the expected behaviour of a well-trained network: the model has learned to associate class-discriminative features (the shape of wings, the texture of fur) with its prediction. Background regions (sky, grass, walls) appear mostly blue, confirming that the network is not relying on context to make its decision for these samples.

Analytical Question 2.3 – Incorrect Prediction Heatmaps

For the two incorrectly classified images the heatmap reveals the source of the error. Common failure patterns observed include:

- **Background confusion** – The network's attention falls on the background (e.g., a large sky region) rather than on the object. If a background colour or texture is more diagnostic of the wrong class in the training set, the model exploits it.
- **Part confusion** – The heatmap highlights a localised object part that resembles a part of the predicted (wrong) class. For example, a ship's prow might activate the same filters as an airplane nose, causing a misclassification.

- **Scale or occlusion** – When the object is small, occluded, or at an unusual angle, the model may fail to activate the right feature maps and instead rely on whatever contextual cues are available, leading to an incorrect decision.

In all these cases, GradCAM makes the misclassification **interpretable**: instead of just knowing the model was wrong, we can see *why* it was wrong and which part of the image it was focused on when it made its error.

Summary of Results

Task	Metric	Value
1A – MNIST CNN	Total parameters	8,650
1A – MNIST CNN	Final test accuracy	**97.31%**
1B – C-MNIST	Biased test accuracy	**96.75%**
1B – C-MNIST	Unbiased test accuracy	**36.64%**
1B – C-MNIST	Performance drop	60.11 pp
2A – STL-10	Trainable parameters (head only)	5,130
2A – STL-10	Test accuracy (frozen backbone)	**94.92%**